



## Technical Code Walkthrough

Generated: January 17, 2026

### Overview

Watchdog AI is a **data quality validation and curation pipeline** designed to clean AI training datasets. It uses **rule-based NLP detection** (no external LLM dependencies) and integrates with the **Climatiq API** for carbon footprint tracking.

### The 4-Step Pipeline:

| Step | Module                  | Purpose                              |
|------|-------------------------|--------------------------------------|
| 1    | Misinformation Detector | Remove high-risk/misleading content  |
| 2    | Quality Scorer          | Filter low-quality items             |
| 3    | Redundancy Detector     | Remove duplicates (exact + semantic) |
| 4    | Sustainability Tracker  | Calculate carbon footprint savings   |

# 1. Misinformation Detector

**File:** src/misinformation\_detector.py

■ **KEY POINT:** This module uses regex-based pattern matching — NOT a machine learning model or LLM. It's fast, deterministic, and has no API costs.

## Core Functions:

| Function                       | Purpose                                                |
|--------------------------------|--------------------------------------------------------|
| detect_generic_content()       | Calculates "vagueness score" using filler words count  |
| detect_clickbait()             | Scans for phrases like "you won't believe"             |
| detect_excessive_caps()        | Calculates uppercase ratio (>60% = high risk)          |
| detect_excessive_punctuation() | Counts !!! and ??? patterns                            |
| check_source_credibility()     | Boosts .gov/.edu, penalizes blog domains               |
| analyze_text()                 | Main entry: combines all signals with weighted scoring |
| batch_analyze()                | Processes multiple texts in a loop                     |

## Weighted Scoring System:

| Signal           | Weight   | Description                                     |
|------------------|----------|-------------------------------------------------|
| Toxicity         | 40%      | Highest weight - triggers high risk immediately |
| Pattern Match    | 35%      | Conspiracy, misinformation phrases              |
| Clickbait        | 25%      | Sensationalist language                         |
| Caps/Punctuation | 10% each | Formatting red flags                            |
| Generic Content  | 10%      | Vague, low-value text                           |
| Source           | 10%      | Domain credibility                              |

## Risk Level Thresholds:

- **High Risk:** Score  $\geq 0.60$  OR  $\geq 3$  flags OR toxicity detected → FILTERED OUT
- **Medium Risk:** Score  $\geq 0.35$  OR  $\geq 1$  flag
- **Low Risk:** Clean content - kept in dataset

## 2. Data Quality Scorer

File: src/quality\_scorer.py

■ **KEY POINT:** Evaluates text structural quality — not meaning. Uses statistical metrics without any ML model to determine if data is well-formed for training.

### Core Functions:

| Function                                      | Purpose                                                          |
|-----------------------------------------------|------------------------------------------------------------------|
| <code>_calculate_text_completeness()</code>   | Checks required (text) + optional fields (title, source, etc.)   |
| <code>_calculate_text_length_score()</code>   | Scores by char count: <20=0.2, 200+=1.0                          |
| <code>_calculate_word_count_score()</code>    | Optimal range: 50-500 words = 1.0                                |
| <code>_calculate_language_quality()</code>    | Capitalization, punctuation, sentence structure                  |
| <code>_calculate_information_density()</code> | KEY: Lexical diversity (unique/total words)                      |
| <code>_calculate_spam_indicators()</code>     | Detects "SHOCKING", "FREE", "CLICK HERE"                         |
| <code>score_data()</code>                     | Main entry: returns overall_score, quality_level, recommendation |

### Quality Scoring Weights:

| Metric              | Weight               |
|---------------------|----------------------|
| Information Density | 28% (most important) |
| Language Quality    | 25%                  |
| Word Count          | 13%                  |
| Completeness        | 12%                  |
| Spam Check          | 12%                  |
| Text Length         | 10%                  |

### Quality Levels & Recommendations:

- **High ( $\geq 0.80$ ):** "Excellent quality - keep"
- **Medium ( $\geq 0.60$ ):** "Good quality - minor review"
- **Low ( $\geq 0.40$ ):** "Fair quality - review before keeping"
- **Very Low ( $< 0.40$ ):** "Poor quality - consider removing" → FILTERED OUT

### 3. Redundancy Detector

**File:** src/redundancy\_detector.py

■ **KEY POINT:** Uses TF-IDF vectorization + cosine similarity from scikit-learn for semantic duplicate detection. Also does exact-match via MD5 hashing. No LLM required.

#### Core Functions:

| Function                                | Purpose                                         |
|-----------------------------------------|-------------------------------------------------|
| <code>_normalize_text()</code>          | Lowercase, strip whitespace, remove punctuation |
| <code>_compute_hash()</code>            | MD5 hash for O(1) exact duplicate lookup        |
| <code>find_exact_duplicates()</code>    | Hash-based matching, groups duplicates          |
| <code>find_semantic_duplicates()</code> | TF-IDF + Cosine Similarity (scikit-learn)       |
| <code>find_duplicates()</code>          | Main entry: supports exact, semantic, or both   |

#### TF-IDF Configuration:

```
TfidfVectorizer(max_features=1000, stop_words='english', ngram_range=(1,2))
```

- **max\_features=1000:** Cap vocabulary size for efficiency
- **stop\_words='english':** Remove common words (the, is, at)
- **ngram\_range=(1,2):** Word + bigram features for context
- **Default threshold:** 85% similarity = duplicate

## 4. Sustainability Tracker

**File:** src/sustainability\_tracker.py

■ **KEY POINT:** Integrates with the Climatiq API to calculate real carbon emissions. Falls back to regional averages if API unavailable.

### Core Functions:

| Function                                           | Purpose                                                     |
|----------------------------------------------------|-------------------------------------------------------------|
| <code>_get_carbon_intensity_from_climatiq()</code> | API call to Climatiq /estimate endpoint                     |
| <code>calculate_data_size_mb()</code>              | Recursively calculates UTF-8 byte size                      |
| <code>track_operation()</code>                     | Logs energy/carbon for text processing, inference, training |
| <code>calculate_savings()</code>                   | KEY: Computes environmental savings from data reduction     |
| <code>get_session_summary()</code>                 | Returns totals + equivalencies (trees, car miles)           |

### Energy Metrics:

| Operation                  | Energy (kWh) |
|----------------------------|--------------|
| Text Processing (per MB)   | 0.00025      |
| Inference (per 1K tokens)  | 0.000004     |
| Model Training (per epoch) | 0.5          |

### Carbon Intensity Fallbacks:

| Region         | kg CO <sub>2</sub> /kWh |
|----------------|-------------------------|
| Global Average | 0.475                   |
| US Average     | 0.386                   |
| EU Average     | 0.295                   |
| Renewable      | 0.05                    |

## 5. Dataset Processor (Orchestration)

File: src/dataset\_processor.py

■ **KEY POINT:** This is the orchestration layer that runs the full 4-step pipeline on entire datasets (CSV, JSON, JSONL).

### Core Functions:

| Function            | Purpose                                                    |
|---------------------|------------------------------------------------------------|
| process_dataframe() | Main pipeline: runs all 4 steps, returns cleaned DataFrame |
| process_csv()       | Load CSV and process                                       |
| process_json()      | Handle JSON array or single object                         |
| process_jsonl()     | Load newline-delimited JSON                                |
| save_results()      | Export as CSV/JSON/JSONL + statistics                      |

### Default Parameters:

- **quality\_threshold = 0.5:** Minimum quality score to keep
- **remove\_high\_risk = True:** Filter misinformation
- **remove\_duplicates = True:** Remove redundant entries
- **similarity\_threshold = 0.85:** 85% similarity = duplicate

## 6. Flask REST API

File: api/app.py

■ **KEY POINT:** Production-ready REST API with CORS support, exposing all modules as HTTP endpoints.

### API Endpoints:

| Endpoint       | Method | Purpose                         |
|----------------|--------|---------------------------------|
| /              | GET    | API info and version            |
| /health        | GET    | Health check for load balancers |
| /analyze       | POST   | Single text analysis            |
| /analyze/batch | POST   | Batch text analysis             |
| /quality       | POST   | Quality scoring only            |
| /duplicates    | POST   | Find duplicates in text list    |

|                 |      |                          |
|-----------------|------|--------------------------|
| /process        | POST | Full pipeline on dataset |
| /sustainability | POST | Calculate carbon savings |

# Key Selling Points

## 1. No External LLM Dependencies

- All detection uses rule-based regex patterns and scikit-learn TF-IDF
- Zero API costs for core analysis (only Climatiq for sustainability)
- Deterministic, explainable results

## 2. Production-Ready Architecture

- Modular components usable standalone or together
- REST API with CORS for frontend integration
- Supports CSV, JSON, JSONL dataset formats

## 3. Explainable AI

- Every detection provides human-readable explanations
- Transparent scoring with visible weights
- Actionable recommendations (keep, review, remove)

## 4. Sustainability Tracking

- Real carbon footprint calculations via Climatiq API
- Quantifies environmental impact of data reduction
- Converts to relatable metrics (trees planted, car miles)

## 5. Computational Efficiency

- TF-IDF vectorization is  $O(n)$  for document count
- MD5 hashing provides  $O(1)$  exact duplicate lookup
- Processes 1000s of items per second on standard hardware